

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Конструирование моделей на базе предобученных нейронных сетей»

Выполнил:

Студент 4 курса

Группы ИИ-26

Кушнеревич Е.А

Проверила:

Андренко К.В.

Цель работы: осуществлять обучение НС, сконструированных на базе предобученных архитектур НС

Ход работы:

1. Для заданной выборки и архитектуры предобученной нейронной организовать процесс обучения НС, предварительно изменив структуру слоев, в соответствии с предложенной выборкой. Использовать тот же оптимизатор, что и в ЛР №1. Построить график изменения ошибки и оценить эффективность обучения на тестовой выборке;
2. Сравнить полученные результаты с результатами, полученными на кастомных архитектурах из ЛР №1;
3. Ознакомиться с state-of-the-art результатами для предлагаемых выборок (по материалам в сети Интернет). Сделать выводы о результатах обучения НС из п. 1 и 2;
4. Реализовать визуализацию работы предобученной СНС и кастомной (из ЛР 1). Визуализация осуществляется посредством выбора и подачи на сеть произвольного изображения (например, из сети Интернет) с отображением результата классификации;

В-т	Выборка	Оптимизатор	Предобученная архитектура
8	CIFAR-10	Adam	MobileNet v3

Код программы:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader

import torchvision
import torchvision.transforms as transforms

import numpy as np
import matplotlib.pyplot as plt
import random
from PIL import Image
from collections import defaultdict

from torchvision.models import mobilenet_v3_small, MobileNet_V3_Small_Weights

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Используемое устройство:", device)

CLASSES = ('airplane', 'automobile', 'bird', 'cat', 'deer',
            'dog', 'frog', 'horse', 'ship', 'truck')

MEAN = (0.4914, 0.4822, 0.4465)
STD = (0.2470, 0.2435, 0.2616)

transform_train = transforms.Compose([
```

```

        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(MEAN, STD),
    ])

transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
])

train_set = torchvision.datasets.CIFAR10(root='./data', train=True,
                                         download=True, transform=transform_train)
test_set = torchvision.datasets.CIFAR10(root='./data', train=False,
                                         download=True, transform=transform_test)

BATCH_SIZE = 128
train_loader = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)
test_loader = DataLoader(test_set, batch_size=256, shuffle=False, num_workers=2)

print(f"Train: {len(train_set)}, Test: {len(test_set)}, размер изображения: {train_set[0][0].shape}")

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        self.fc1 = nn.Linear(128 * 4 * 4, 256)
        self.fc2 = nn.Linear(256, num_classes)
        self.dropout = nn.Dropout(0.4)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

model = SimpleCNN().to(device)
n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(model)
print(f"Обучаемых параметров: {n_params:,}")

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)
N_EPOCHS = 10

history = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc": []}

def run_epoch(loader, train=True):
    model.train() if train else model.eval()
    total_loss, correct, total = 0.0, 0, 0
    context = torch.enable_grad() if train else torch.no_grad()

```

```

with context:
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        if train:
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            if train:
                loss.backward()
                optimizer.step()
            total_loss += loss.item() * images.size(0)
            correct += (outputs.argmax(1) == labels).sum().item()
            total += labels.size(0)
    return total_loss / total, correct / total

for epoch in range(1, N_EPOCHS + 1):
    train_loss, train_acc = run_epoch(train_loader, train=True)
    test_loss, test_acc = run_epoch(test_loader, train=False)
    history["train_loss"].append(train_loss)
    history["train_acc"].append(train_acc)
    history["test_loss"].append(test_loss)
    history["test_acc"].append(test_acc)
    print(f"Эпоха {epoch:2d}/{N_EPOCHS} | train loss {train_loss:.4f} acc
{train_acc*100:5.2f}% | "
          f"test loss {test_loss:.4f} acc {test_acc*100:5.2f}%")

custom_model = model

IMAGENET_MEAN = (0.485, 0.456, 0.406)
IMAGENET_STD = (0.229, 0.224, 0.225)

transform_train_pt = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])

transform_test_pt = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])

train_set_pt = torchvision.datasets.CIFAR10(root='./data', train=True,
                                             download=True, transform=transform_train_pt)
test_set_pt = torchvision.datasets.CIFAR10(root='./data', train=False,
                                             download=True, transform=transform_test_pt)

BATCH_SIZE_PT = 64
train_loader_pt = DataLoader(train_set_pt, batch_size=BATCH_SIZE_PT, shuffle=True,
                             num_workers=2)
test_loader_pt = DataLoader(test_set_pt, batch_size=128, shuffle=False, num_workers=2)

print(f"Train: {len(train_set_pt)}, Test: {len(test_set_pt)}, размер изображения:
{train_set_pt[0][0].shape}")

weights = MobileNet_V3_Small_Weights.IMAGENET1K_V1
pretrained_model = mobilenet_v3_small(weights=weights)

```

```

for param in pretrained_model.features.parameters():
    param.requires_grad = False

in_features = pretrained_model.classifier[-1].in_features
pretrained_model.classifier[-1] = nn.Linear(in_features, len(CLASSES))

pretrained_model = pretrained_model.to(device)

n_params_total = sum(p.numel() for p in pretrained_model.parameters())
n_params_trainable = sum(p.numel() for p in pretrained_model.parameters() if p.requires_grad)
print(pretrained_model.classifier)
print(f"Всего параметров: {n_params_total:,}")
print(f"Обучаемых параметров (только классификатор): {n_params_trainable:,}")

criterion_pt = nn.CrossEntropyLoss()
optimizer_pt = optim.Adam(filter(lambda p: p.requires_grad, pretrained_model.parameters()),
                             lr=1e-3)

N_EPOCHS_PT = 5
history_pt = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc": []}

def run_epoch_pt(loader, train=True):
    pretrained_model.train() if train else pretrained_model.eval()
    total_loss, correct, total = 0.0, 0, 0
    context = torch.enable_grad() if train else torch.no_grad()
    with context:
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            if train:
                optimizer_pt.zero_grad()
                outputs = pretrained_model(images)
                loss = criterion_pt(outputs, labels)
                if train:
                    loss.backward()
                    optimizer_pt.step()
                total_loss += loss.item() * images.size(0)
                correct += (outputs.argmax(1) == labels).sum().item()
                total += labels.size(0)
    return total_loss / total, correct / total

for epoch in range(1, N_EPOCHS_PT + 1):
    train_loss, train_acc = run_epoch_pt(train_loader_pt, train=True)
    test_loss, test_acc = run_epoch_pt(test_loader_pt, train=False)
    history_pt["train_loss"].append(train_loss)
    history_pt["train_acc"].append(train_acc)
    history_pt["test_loss"].append(test_loss)
    history_pt["test_acc"].append(test_acc)
    print(f"Эпоха {epoch:2d}/{N_EPOCHS_PT} | train loss {train_loss:.4f} acc {train_acc*100:5.2f}% | "
          f"test loss {test_loss:.4f} acc {test_acc*100:5.2f}%")

epochs_range_pt = range(1, N_EPOCHS_PT + 1)
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].plot(epochs_range_pt, history_pt["train_loss"], label="Train loss", marker='o',
             markersize=3)
axes[0].plot(epochs_range_pt, history_pt["test_loss"], label="Test loss", marker='o',
             markersize=3)
axes[0].set_xlabel("Эпоха"); axes[0].set_ylabel("Loss")
axes[0].set_title("MobileNetV3: изменение ошибки"); axes[0].legend(); axes[0].grid(alpha=0.3)

```

```

axes[1].plot(epochs_range_pt, [a*100 for a in history_pt["train_acc"]], label="Train acc",
marker='o', markersize=3)
axes[1].plot(epochs_range_pt, [a*100 for a in history_pt["test_acc"]], label="Test acc",
marker='o', markersize=3)
axes[1].set_xlabel("Эпоха"); axes[1].set_ylabel("Accuracy, %")
axes[1].set_title("MobileNetV3: изменение точности"); axes[1].legend();
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

print(f"Итоговая точность MobileNetV3 на тесте: {history_pt['test_acc'][-1]*100:.2f}%")

print("Сравнение результатов:")
print(f"Кастомная ЧСЧ: {history['test_acc'][-1]*100:.2f}% ({n_params:,} параметров, все обучаемые)")
print(f"MobileNetV3: {history_pt['test_acc'][-1]*100:.2f}% ({n_params_trainable:,} обучаемых параметров из {n_params_total:,})")

def predict_with_both(path):
    img = Image.open(path).convert('RGB')

    preprocess_custom = transforms.Compose([
        transforms.Resize((32, 32)),
        transforms.ToTensor(),
        transforms.Normalize(MEAN, STD),
    ])
    preprocess_pt = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    ])

    custom_model.eval()
    pretrained_model.eval()

    with torch.no_grad():
        probs_custom = F.softmax(custom_model(preprocess_custom(img).unsqueeze(0).to(device)),
dim=1).cpu().numpy().flatten()
        probs_pt = F.softmax(pretrained_model(preprocess_pt(img).unsqueeze(0).to(device)),
dim=1).cpu().numpy().flatten()

    pred_custom = int(np.argmax(probs_custom))
    pred_pt = int(np.argmax(probs_pt))

    fig, axes = plt.subplots(1, 3, figsize=(15, 4), gridspec_kw={'width_ratios': [1, 1.2, 1.2]})
    axes[0].imshow(img); axes[0].axis('off'); axes[0].set_title("Входное изображение")

    axes[1].barh(CLASSES, probs_custom * 100, color=['red' if i == pred_custom else 'steelblue' for i in range(10)])
    axes[1].set_xlim(0, 100); axes[1].invert_yaxis()
    axes[1].set_title(f"Кастомная ЧСЧ: {CLASSES[pred_custom]}")
    axes[1].set_xlabel("Вероятность, %")

    axes[2].barh(CLASSES, probs_pt * 100, color=['green' if i == pred_pt else 'steelblue' for i in range(10)])
    axes[2].set_xlim(0, 100); axes[2].invert_yaxis()
    axes[2].set_title(f"MobileNetV3: {CLASSES[pred_pt]}")

```

```
axes[2].set_xlabel("Вероятность, %")
```

```
plt.tight_layout()
plt.show()
return pred_custom, pred_pt
```

```
predict_with_both(r'C:\Users\User\Desktop\News\doggie.jpg')
```

Результат тестирования:

Используемое устройство: cuda

100%|██████████| 170M/170M [29:33<00:00, 96.2kB/s]

Train: 50000, Test: 10000, размер изображения: torch.Size([3, 32, 32])

```
SimpleCNN(
  (conv1): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
  ceil_mode=False)
  (fc1): Linear(in_features=2048, out_features=256, bias=True)
  (fc2): Linear(in_features=256, out_features=10, bias=True)
  (dropout): Dropout(p=0.4, inplace=False)
)
```

Обучаемых параметров: 620,362

Эпоха	1/10		train loss	1.6676	acc	38.52%		test loss	1.3020	acc	52.78%
Эпоха	2/10		train loss	1.3243	acc	52.13%		test loss	1.0848	acc	60.85%
Эпоха	3/10		train loss	1.1527	acc	58.72%		test loss	0.9720	acc	65.42%
Эпоха	4/10		train loss	1.0445	acc	62.73%		test loss	0.8778	acc	68.85%
Эпоха	5/10		train loss	0.9676	acc	65.86%		test loss	0.8219	acc	70.79%
Эпоха	6/10		train loss	0.9063	acc	68.16%		test loss	0.7699	acc	72.90%
Эпоха	7/10		train loss	0.8600	acc	70.01%		test loss	0.7715	acc	73.26%
Эпоха	8/10		train loss	0.8194	acc	71.26%		test loss	0.7303	acc	74.79%
Эпоха	9/10		train loss	0.7899	acc	72.43%		test loss	0.6917	acc	75.90%
Эпоха	10/10		train loss	0.7581	acc	73.54%		test loss	0.6999	acc	76.16%

MobileNet v3:

Train: 50000, Test: 10000, размер изображения: torch.Size([3, 224, 224])

Downloading: "https://download.pytorch.org/models/mobilenet_v3_small-047dcff4.pth" to /root/.cache/torch/hub/checkpoints/mobilenet_v3_small-047dcff4.pth

100%|██████████| 9.83M/9.83M [00:00<00:00, 202MB/s]

```
Sequential(
  (0): Linear(in_features=576, out_features=1024, bias=True)
  (1): Hardswish()
  (2): Dropout(p=0.2, inplace=True)
  (3): Linear(in_features=1024, out_features=10, bias=True)
)
```

Всего параметров: 1,528,106

Обучаемых параметров (только классификатор): 601,098

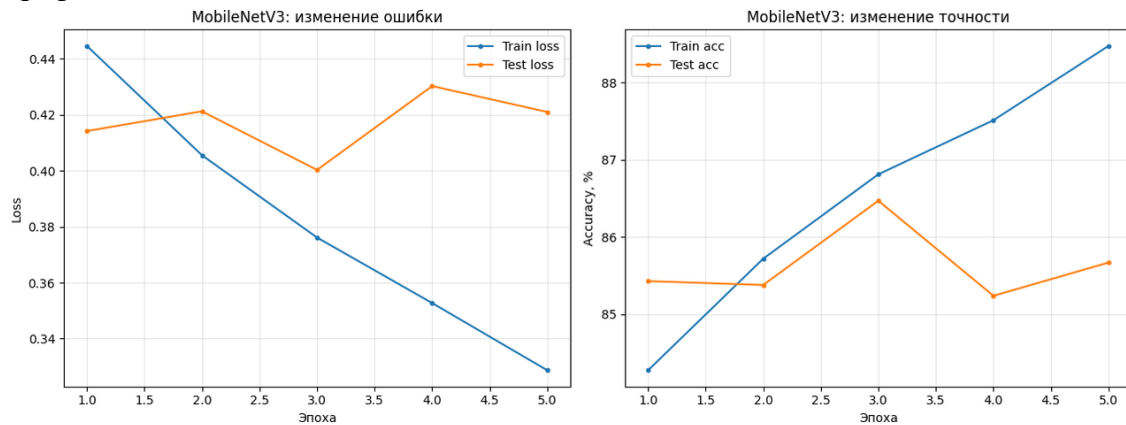
Эпоха	1/5		train loss	0.4446	acc	84.28%		test loss	0.4142	acc	85.43%
Эпоха	2/5		train loss	0.4056	acc	85.72%		test loss	0.4213	acc	85.38%
Эпоха	3/5		train loss	0.3761	acc	86.81%		test loss	0.4003	acc	86.47%
Эпоха	4/5		train loss	0.3527	acc	87.51%		test loss	0.4303	acc	85.24%
Эпоха	5/5		train loss	0.3287	acc	88.47%		test loss	0.4210	acc	85.67%

Сравнение результатов:

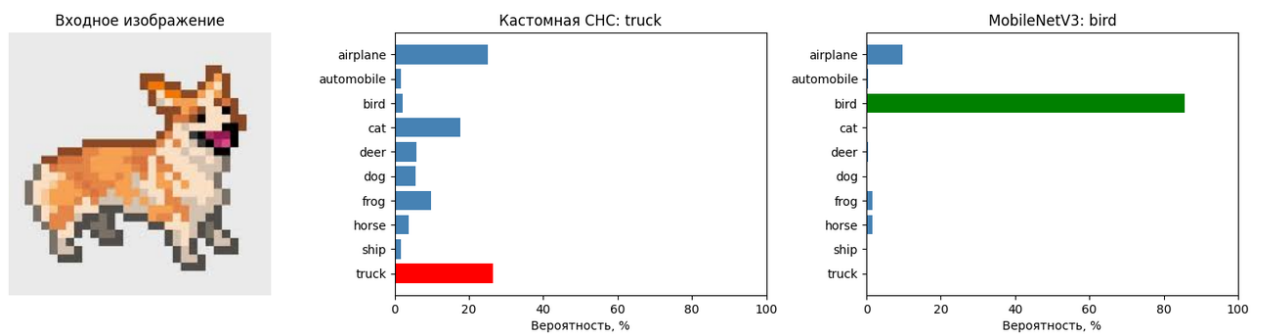
Кастомная СНС: 76.16% (620,362 параметров, все обучаемые)

MobileNetV3: 85.67% (601,098 обучаемых параметров из 1,528,106)

График изменения ошибки:



Итоговая точность MobileNetV3 на тесте: 85.67%



Готовые, предобученные сети – это мощный инструмент: они дают гораздо более высокую точность за куда меньшее время обучения, потому что не приходится учить всё с нуля. Но это не бесплатный выигрыш: такая сеть быстрее "зазубривает" небольшой обучающий набор (переобучается), если не добавить дополнительные меры защиты (аугментацию, регуляризацию скорости обучения), и может вести себя обманчиво уверенно там, где честная простая модель просто скажет "не уверена". Иными словами: одолжить чужой опыт – это здорово, но пользоваться им нужно аккуратно, а не просто взять и ожидать, что всё само получится идеально.

Вывод: Проведя данную лабораторную работу я осуществил обучение НС, сконструированных на базе предобученных архитектур НС